

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA SAU ĐẠI HỌC



BÁO CÁO BÀI TẬP LỚN

Các Hệ Thống Phân Tán

HỆ THỐNG ĐẶT MÓN ĂN THEO KIẾN TRÚC MICROSERVICE

(Chủ đề 4: Microservice — REST API & Message Queue)

Giảng viên: TS. Kim Ngọc Bách

Lớp: M25CQHT02-B

Nhóm: 5

01. Lê Thiện Văn MHV: B25CHHT120

02. Lê Thiện Đức MHV: B25CHHT085

03. Vũ Hải Nam MHV: B25CHHT107

Hà nội - 2026

Table of Contents

1. Giới thiệu & mô tả bài toán	3
1.1. Bối cảnh	3
1.2. Bốn vai trò (actor)	3
1.3. Phạm vi & mục tiêu học thuật.....	3
2. Cơ sở lý thuyết hệ phân tán áp dụng.....	3
3. Kiến trúc tổng thể	4
4. Chi tiết 10 microservice.....	5
5. Thiết kế RESTful API	6
6. Giao tiếp đồng bộ — Saga đặt hàng (gRPC).....	7
7. Giao tiếp bất đồng bộ — Message Queue (Kafka + Outbox).....	8
7.1. Transactional Outbox + consumer at-least-once (đảm bảo không mất sự kiện)..	8
7.2. Catalog sự kiện & ánh xạ hành động	9
7.3. Luồng tiêu biểu dùng MQ (đáp ứng yêu cầu bắt buộc của đề).....	9
8. Mô hình dữ liệu & lược đồ CSDL (ERD)	10
9. Nhất quán dữ liệu & giao dịch phân tán	11
10. Bảo mật & phân quyền (RBAC/JWT)	12
11. Quan sát, ghi log & chịu lỗi	12
12. Kiểm thử & đánh giá.....	13
13. Triển khai & hướng dẫn chạy	14
14. Đối chiếu yêu cầu đề bài	15
15. Hạn chế & hướng phát triển.....	17
16. Kết luận.....	17
17. Phụ lục.....	18
17.1. Bảng trạng thái (enum) quan trọng.....	18
17.2. Catalog topic Kafka.....	18
17.3. Tham chiếu mã nguồn chính	18

1. Giới thiệu & mô tả bài toán

1.1. Bối cảnh

VeloxFood là nền tảng đặt đồ ăn trực tuyến phục vụ trong khuôn viên trường, theo mô hình marketplace nhiều cửa hàng: nhiều quán cùng bán, khách đặt, tài xế giao tới tận Phòng trong toà nhà. Bài toán mang đầy đủ tính chất của một hệ phân tán: nhiều miền nghiệp vụ độc lập (định danh, danh mục, đơn hàng, thanh toán, giao vận, đánh giá, thông báo, báo cáo) phối hợp xử lý một giao dịch nghiệp vụ trải dài nhiều bước.

1.2. Bốn vai trò (actor)

Vai trò	Chức năng chính
Khách hàng	Đăng ký/đăng nhập (OTP), duyệt & tìm cửa hàng/món, giỏ hàng theo từng quán, đặt đơn (chọn địa điểm Toà→Tầng→Phòng, giờ nhận, voucher, COD/Ví/MoMo), theo dõi đơn realtime, đặt lại, đánh giá, ví điện tử
Chủ cửa hàng	Đăng ký bán hàng (admin duyệt), quản lý cửa hàng: danh mục, thực đơn (ảnh, tùy chọn, combo), phí giao theo cấp địa điểm, giờ hoạt động (admin duyệt), khuyến mãi; xử lý đơn (xác nhận→chuẩn bị→sẵn sàng); xem doanh thu, đánh giá
Tài xế	Đăng ký (upload giấy tờ + admin duyệt), nhận đơn, lấy hàng, giao, báo sự cố (kèm ảnh), xem thu nhập
Quản trị / Super-admin	Quản lý người dùng, duyệt chủ quán & tài xế, tạo cửa hàng, quản lý cây vị trí, đối soát & chi trả, xử lý sự cố, xem báo cáo vận hành

1.3. Phạm vi & mục tiêu học thuật

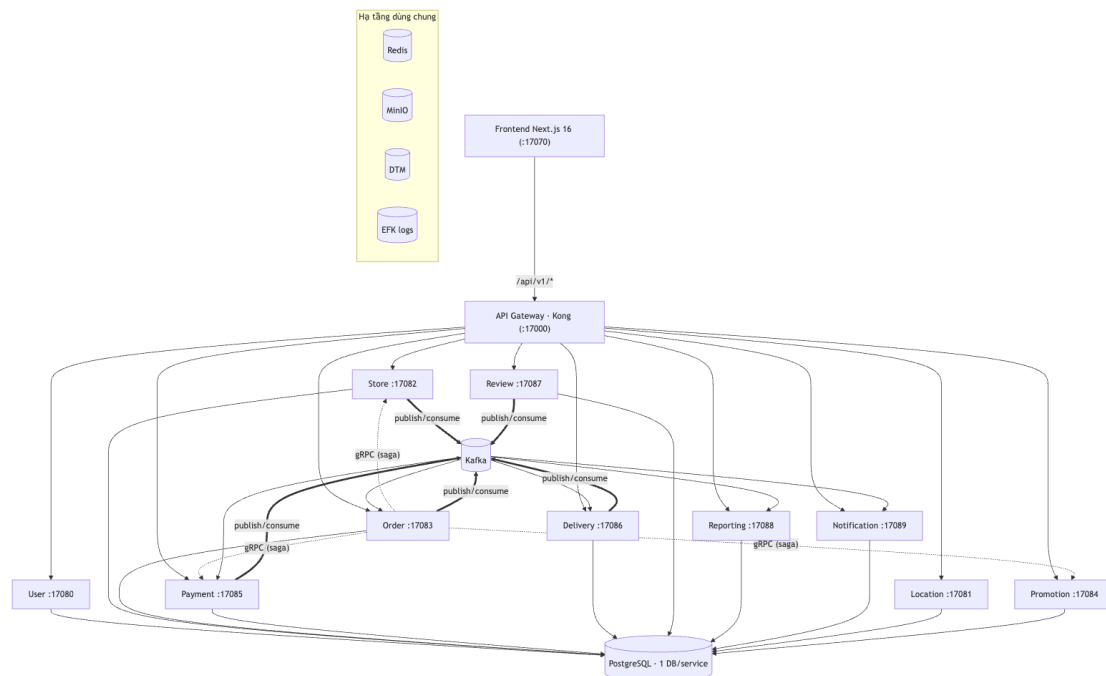
Mục tiêu là vận dụng các kiến thức hệ phân tán: phân tách dịch vụ, giao tiếp liên tiến trình (đồng bộ REST/gRPC + bất đồng bộ qua hàng đợi), giao dịch phân tán (saga), nhất quán dữ liệu, tính sẵn sàng & chịu lỗi, quan sát hệ thống.

2. Cơ sở lý thuyết hệ phân tán áp dụng

Khái niệm	Áp dụng trong VeloxFood
Microservice / phân rã theo miền	10 service theo bounded-context, mỗi service một DB (database-per-service) → tách rời triển khai & dữ liệu
Giao tiếp đồng bộ	REST (client↔gateway↔service) + gRPC (service↔service trong saga, cần kết quả tức thời)
Giao tiếp bất đồng bộ	Message Queue (Kafka) cho lan truyền sự kiện, giảm phụ thuộc thời gian (temporal decoupling)
Giao dịch phân tán	Saga (orchestration) do DTM điều phối, thay cho 2PC: chuỗi giao dịch cục bộ + bù trừ (compensation); sub-transaction barrier (bằng dtm_barrier) chống null-compensation và duplicate branch call
Nhất quán	<i>Nhất quán mạnh</i> trong saga (gRPC + khoá <i>FOR UPDATE</i>); <i>nhất quán cuối</i> (eventual) giữa các service qua sự kiện
Định lý CAP	Hệ ưu tiên AP ở luồng bất đồng bộ (sẵn sàng + chịu phân mảnh, nhất quán cuối); CP cục bộ ở thao tác tiền/kho trong saga

Khái niệm	Áp dụng trong VeloxFood
Độ tin cậy truyền tin	Transactional Outbox (không mất sự kiện) + consumer at-least-once (commit offset sau khi xử lý thành công, retry 3 lần backoff, Dead-Letter Queue <topic>.dlq) + idempotent consumer (processed_events)
Chịu lỗi	Outbox retry + consumer retry/DLQ; circuit breaker (gobreaker) + timeout 5s + retry cho mọi gRPC liên service; bù trừ bền vững qua pending_compensations; graceful degrade (Redis/MinIO/downstream lỗi không sập); graceful shutdown; atomic claim chống tranh chấp; rate-limit tại gateway
Quan sát (observability)	Log JSON có cấu trúc + trace_id lan truyền (correlation-id) → EFK tập trung; tracing phân tán OpenTelemetry → Jaeger; metrics Prometheus + Grafana
API Gateway / Service discovery	Kong định tuyến tập trung, là điểm vào duy nhất; ẩn topology nội bộ

3. Kiến trúc tổng thể



Khung dùng chung *pkg/app*: mỗi service khởi tạo qua *app.New("<svc>")* → nạp config (Viper), logger *slog* (JSON), GORM Postgres (pool 25/10), auto-migrate khi khởi động (golang-migrate, advisory lock — nhiều replica start cùng lúc không đua nhau), Redis (degrade khi lỗi), gRPC server + Gin HTTP (*/health* liveness + */readyz* readiness ping DB/Redis, middleware log + trace-id), graceful shutdown (HTTP drain 15s → gRPC → consumer → DB/Redis). Tách lớp clean architecture: *handler* → *usecase* → *repository (interface)* ← *persistence (GORM)*; entity là leaf.

Stack công nghệ

Hạng mục	Nội dung
Loại hệ thống	Web, nhiều cửa hàng (multi-vendor), 4 vai trò người dùng
Backend	Go 1.25 — 10 microservice (single module project), Gin (REST) + gRPC
Frontend	Next.js 16 (App Router), React 19, TypeScript, TanStack Query
Message Queue	Apache Kafka 3.8 (Transactional Outbox + idempotent consumer)

Hạng mục	Nội dung
API Gateway	Kong 3.7 (declarative)
CSDL	PostgreSQL — mỗi service một database riêng
Hạ tầng phụ	Redis (cache), MinIO (ảnh — bucket public + bucket private cho KYC), DTM (saga), EFK (log tập trung), Docker Compose
Quy mô	180 REST endpoint · 9 Kafka topic (+DLQ) · ~30 loại sự kiện · 21 consumer group · 40 permission

4. Chi tiết 10 microservice

#	Service	Cổng	DB	Trách nhiệm	Sự kiện phát
1	User	17080	user_db	Định danh, JWT, OTP, RBAC (role/permission có scope), onboard chủ quán, đăng ký tài xế, audit log	<i>user.activated/suspended/role_assigned</i>
2	Location	17081	location_db	Cây Toà→Tầng→Phòng, địa điểm đã lưu của khách	—
3	Store	17082	store_db	Cửa hàng, danh mục, thực đơn, tùy chọn/combo, phí giao theo cấp, giờ hoạt động, tìm món toàn hệ thống (filter/sort), yêu thích quán & món	<i>store.created/status_changed</i>
4	Order	17083	order_db	Giỏ hàng, saga đặt hàng, vòng đời + lịch sử trạng thái, hủy/đặt-lại	<i>order.placed/confirmed/preparing/ready/ready_pickup/delivered/completed/cancelled/rejected</i>
5	Promotion	17084	promotion_db	Voucher theo cửa hàng, validate, áp dụng (participant saga)	(qua gRPC)
6	Payment	17085	payment_db	Ví — sổ cái kép, COD/Ví/MoMo, đối soát + chi trả	<i>payment.captured/refunded, wallet.topped_up/debited/refunded, payout.settled</i>
7	Delivery	17086	delivery_db	Atomic claim, vòng đời giao, batch ≤5, sự cố	<i>delivery.claimed/delivering/delivered/incident_reported/status_changed</i>
8	Review	17087	review_db	Đánh giá Cửa hàng/Món/Tài xế (kèm ảnh + phân bố sao), trả lời, kiểm duyệt	<i>review.created/reported</i>
9	Reporting	17088	reporting_db	CQRS read-model từ sự kiện → analytics admin, doanh thu quán	— (chỉ tiêu thụ)
10	Notification	17089	notification_db	Thông báo in-app/email/push (FCM) từ mọi sự kiện	— (chỉ tiêu thụ)

5. Thiết kế RESTful API

Nguyên tắc: tài nguyên danh từ số nhiều, động từ HTTP, prefix */api/v1*, body snake_case, mã trạng thái chuẩn (200/201/400/401/403/404/409/429/500), bao response thống nhất *{status, message, data, error}*, phân trang *{items, total, page}*. Toàn hệ thống 180 endpoint route qua Kong; 3 lớp middleware: *AuthMiddleware* (JWT) → *perm("...")* (RBAC global) / *vendorPerm("...")* (RBAC scope cửa hàng) / *AdminPermission*. Gateway áp rate-limit theo IP (300 req/phút toàn cục; 10 req/phút cho */auth/** và đăng ký shipper — chống brute-force).

Một số endpoint tiêu biểu (bảng đầy đủ ở mã nguồn *router.go* mỗi service):

User (45): *POST /auth/{register,verify-register,login,refresh,logout,forgot-password,reset-password}*, *GET /auth/google/{login,callback}*, *GET|PATCH /me*, *GET /admin/users*, *POST /admin/users/:id/roles*, *POST /admin/shippers/:id/{approve,reject}*, *POST /admin/vendors/:id/{approve,reject}*, *POST /vendors/onboard*, *POST /shipper/register*, *CRUD /roles & /permissions*.

Store (54): *GET /menu-items/search* (public — hỗ trợ sort=price_asc|price_desc, price_min/max, store_id; mỗi kết quả kèm OpenNow tính từ giờ hoạt động), *GET /stores*, *GET /stores/:id/{menu,categories,ship-fee,hours}*, *GET /me/favorites{/ids} + PUT|DELETE /me/favorites/:type/:id* (yêu thích quán/món, idempotent), *PATCH /stores/:id*, *POST /stores/:id/{categories,menu,option-groups,combos,ship-fees,hours-change,avatar}* (owner-scope), *POST /admin/stores*, *POST /admin/stores/:id/hours-change/:req/{approve,reject}*.

Order (16): *GET /me/carts*, *PUT /me/cart/items*, *POST /orders*, *GET /orders/:id*, *POST /orders/:id/{cancel,reorder}*, *POST /stores/:storeId/orders/:id/{confirm,reject,pickup-verify}*, *PATCH /stores/:storeId/orders/:id/status*.

Payment (9): *POST /payments/momo/ipn* (public S2S), *GET /me/wallet*, *POST /wallet/topup*, *GET /me/store-revenue*, *GET|POST /admin/payouts*, *POST /admin/payouts/:id/execute*, *GET /admin/settlements*.

Delivery (10): *GET /deliveries/available*, *POST /deliveries/:orderId/claim*, *PATCH /deliveries/:orderId/status*, *POST /deliveries/:orderId/{incident,incident-photo}*, *GET /me/deliveries*, *GET /admin/incidents*, *PATCH /admin/incidents/:id/resolve*.

Review (11): *POST /orders/:id/reviews* (kèm photo_urls), *POST /reviews/photos* (upload ảnh đánh giá), *GET /stores/:id/reviews{/summary,/item-summaries,/items/:itemId}* (summary trả thêm histogram sao Counts[1★..5★]; list lọc được rating=1..5), *POST /reviews/:id/{report,reply}*, admin hide/restore.

Trải nghiệm người dùng phía web (Next.js): tìm món có bộ lọc (sắp xếp giá/liên quan, khoảng giá, chỉ quán đang mở) + lịch sử tìm kiếm gần đây; yêu thích quán & món (nút tim optimistic, trang */account/favorites*); đánh giá kèm tối đa 3 ảnh + biểu đồ phân bố sao có thể bấm để lọc; theo dõi đơn có banner giờ nhận dự kiến (đếm ngược, cảnh báo trễ); danh sách quán sắp xếp theo điểm đánh giá; ảnh tối ưu qua

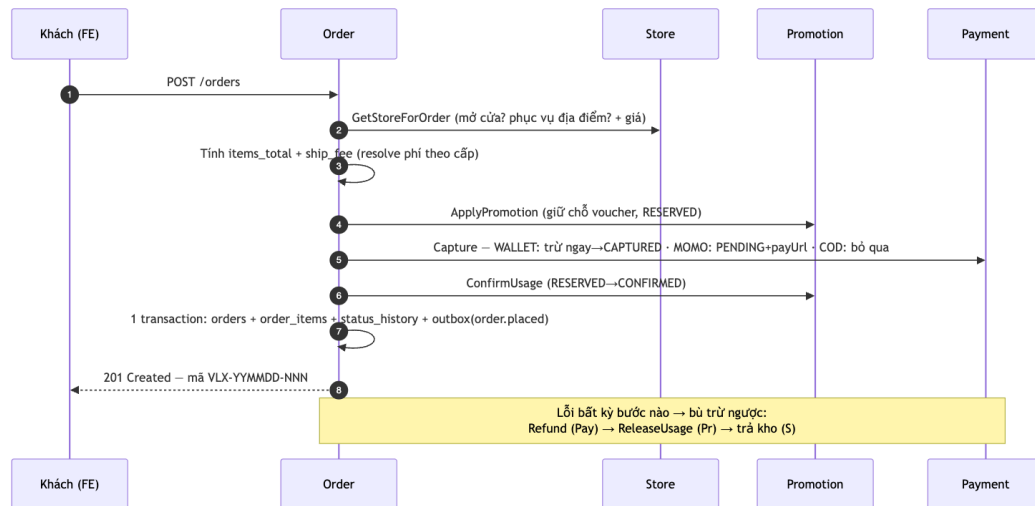
next/image; thông báo thân thiện khi bị rate-limit (429). Reporting (4): *GET /admin/analytics*, *GET /admin/orders/recent*, *GET /stores/:id/{reports,revenue}*. Notification (6): *GET /me/notifications*, *GET /me/notifications/unread-count*, *PATCH .../:id/read*, device-tokens. Location (20): duyệt cây + CRUD admin *buildings/floors/rooms* + */me/locations*. Promotion (5): CRUD */stores/:storeId/promotions* + *POST /promotions/validate*.

6. Giao tiếp đồng bộ — Saga đặt hàng (gRPC)

Đặt hàng là giao dịch trải nhiều service cần nhất quán tức thời (áp voucher, thu tiền) → dùng saga điều phối (orchestration), không dùng 2PC. Engine mặc định là DTM (saga.engine: dtm trong config/order.yaml), chạy chế độ Saga đồng bộ (dtmgrpc.NewSagaGrpc, WaitResult=true): Order mở saga tới DTM server, DTM lần lượt gọi các participant qua gRPC và tự động chạy bù trừ ngược khi một nhánh trả về codes.Aborted. Ba nhánh của saga đặt hàng: (1) Promotion.ApplyPromotion — bù trừ ReleaseUsage (chỉ khi có voucher); (2) Payment.Capture — bù trừ Refund (bỏ qua với COD); (3) Promotion.ConfirmUsage — bù trừ ReleaseUsage. Store không phải participant: chỉ được gọi read-only trước khi mở saga (GetStoreForOrder — kiểm tra quán mở cửa, vùng phục vụ, chốt giá và phí giao) nên không cần bù trừ; hệ cũng không mô hình hoá tồn kho món. Mọi lời gọi gRPC nội bộ đi qua client chuẩn pkg/grpcx với deadline 5 giây và circuit breaker, nên participant treo sẽ fail-fast để saga bù trừ thay vì treo cả chuỗi. Hệ vẫn giữ một engine inline (Order tự điều phối tuần tự) làm phương án dự phòng khi không kết nối được DTM lúc mở saga và để kiểm thử failure-injection. (Mã: services/order/internal/usecase/place_order_dtm.go, place_order_usecase.go, pkg/saga/, pkg/grpcx/client.go.)

Sub-transaction barrier: mỗi participant bọc thao tác nghiệp vụ trong RunWithBarrier — một dòng chặn (gid, branch_id, op) được ghi vào bảng dtm_barrier trong CÙNG transaction với thay đổi nghiệp vụ. Nhờ đó, DTM gọi trùng một nhánh đã thành công sẽ thành no-op (chống trừ tiền hai lần / giữ voucher hai lần), và lời gọi bù trừ cho một hành động chưa từng chạy (null compensation) cũng thành no-op, đồng thời chặn luôn hành động đến muộn sau khi đã bù trừ. Lỗi nghiệp vụ được map sang gRPC codes.Aborted để báo DTM rollback ngay, phân biệt với lỗi hạ tầng (DTM sẽ retry).

Saga thứ hai — huỷ đơn: khi khách huỷ một đơn còn PENDING, Order mở một saga DTM gồm Payment.Refund (nếu đã thanh toán online) và Promotion.ReleaseUsage (nếu có voucher). Saga này không có nhánh bù trừ, vì cả hai thao tác đều idempotent và chỉ tiến (forward-only) — DTM chỉ việc retry cho tới khi thành công. Như vậy hệ minh hoạ hai kiểu saga khác nhau: compensating saga (đặt hàng) và idempotent-retry saga (huỷ đơn). Consumer Kafka order.cancelled vẫn là lưới an toàn hội tụ cuối cùng. (Mã: services/order/internal/usecase/order_cancel_dtm.go.)



Khác biệt theo phương thức thanh toán:

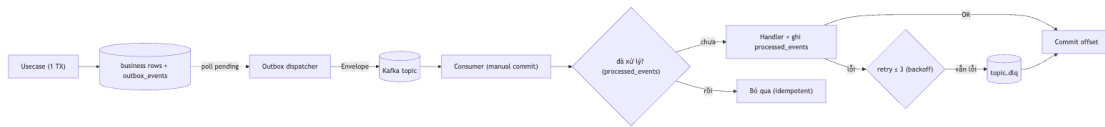
- WALLET: Capture đồng bộ → trừ ví khách ngay, đơn *PAID* lập tức.
- MoMo: Capture trả *payUrl*, đơn *UNPAID/PENDING* tới khi nhận IPN *payment.captured*.
- COD: Không thu lúc đặt; thanh toán được ghi nhận bắt đồng bộ khi giao xong (xem §7) — cho phép Payment & Delivery tách rời.

7. Giao tiếp bất đồng bộ — Message Queue (Kafka + Outbox)

7.1. Transactional Outbox + consumer at-least-once (đảm bảo không mất sự kiện)

Phía phát: mỗi service ghi sự kiện vào bảng `outbox_events` trong cùng transaction với thay đổi nghiệp vụ → một dispatcher nền đọc `status='pending'`, bọc `Envelope {version, event_id, event_type, occurred_at, trace_id, data}`, publish lên Kafka rồi đánh dấu `published` (retry khi lỗi). `Envelope` có trường `version` (hiện "1") và `envelope` cũ chưa có `version` vẫn decode được — cho phép tiến hoá schema sự kiện mà không phá vỡ consumer cũ (có test riêng: `pkg/outbox/envelope_version_test.go`). (Mã: `pkg/outbox/outbox.go`.)

Phía nhận (at-least-once): consumer chỉ commit offset sau khi handler xử lý thành công — handler lỗi được retry tối đa 3 lần (backoff 0.5/1/2s); hết retry, message được đẩy sang Dead-Letter Queue `<topic>.dlq` kèm header truy vết `x-original-topic/partition/offset`, `x-error`, `x-failed-at` (giữ nguyên cả `x-trace-id` gốc) rồi mới commit — không sự kiện nghiệp vụ nào bị mất âm thầm. Trường hợp vừa không xử lý được vừa không đẩy được sang DLQ, consumer cố ý KHÔNG commit và dừng lại ở message đó để bảo toàn thứ tự trong partition thay vì bỏ qua. Bên nhận kiểm tra bảng `processed_events` (PK = `event_id`) để khử trùng lặp → hiệu ứng đúng-một-lần dù giao ít-nhất-một-lần. (Mã: `pkg/messaging/kafka/consumer.go`.)



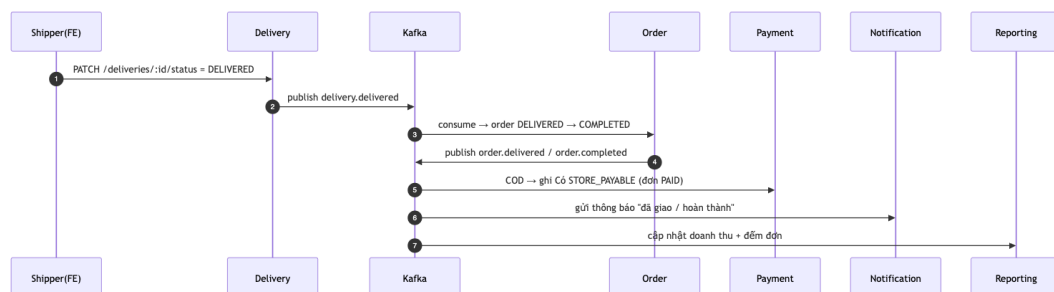
7.2.Catalog sự kiện & ánh xạ hành động

Topic = aggregate của bên phát. Toàn hệ có 9 topic (order, payment, wallet, payout, delivery, store, review, vendor, user), mỗi topic kèm một dead-letter <topic>.dlq, và 21 consumer group. Các luồng chính:

Topic	Sự kiện chính	Bên tiêu thụ → hành động
<i>order.events</i>	<i>order.placed</i>	Notification (báo khách), Reporting (đếm/projection)
	<i>order.ready</i>	Delivery → tạo đơn giao trạng thái AVAILABLE
	<i>order.delivered</i>	Payment → COD: ghi sổ kép Nợ SYSTEM / Có STORE_PAYABLE (chuyển PAID); Order → auto COMPLETED; Review mở khoá đánh giá; Notification
	<i>order.cancelled</i>	Payment (hoàn nếu đã trả online), Promotion (ReleaseUsage), Delivery (huỷ), Reporting, Notification
<i>payment.events</i>	<i>payment.captured/refunded</i>	Order → cập nhật <i>payment_status</i> ; Reporting; Notification
<i>delivery.events</i>	<i>delivery.delivered</i>	Order → DELIVERED→COMPLETED; Notification
<i>wallet.events</i>	<i>wallet.topped_up/debited/refunded</i>	Notification
<i>payout.events</i>	<i>payout.settled</i>	Reporting, Notification
<i>review.events</i>	<i>review.created</i>	Reporting → cập nhật điểm trung bình cửa hàng (<i>store_performance</i>)

7.3.Luồng tiêu biểu dùng MQ (đáp ứng yêu cầu bắt buộc của đề)

Khi tài xế bấm Đã giao:



→ Một sự kiện kích hoạt nhiều service phản ứng độc lập, không gọi trực tiếp lẫn nhau — minh họa rõ giá trị của hàng đợi (decoupling, chịu lỗi từng phần).

8. Mô hình dữ liệu & lược đồ CSDL (ERD)

Mỗi service sở hữu schema riêng (migration *golang-migrate*). Tiền lưu int64 (đơn vị nhỏ nhất, tránh sai số dấu phẩy động). Bảng chính theo service:

- **User:** *users* (id, email/phone unique, password_hash bcrypt, status), *roles*, *permissions*, *user_roles* (scope_type/scope_id), *role_permissions*, *identities* (google/local), *vendor_memberships*, *shipper_profiles* (ảnh + duyệt), *audit_logs*, *otp_codes*, *password_reset_tokens*, *invitations*, *outbox_events*.
- **Location:** *buildings* → *floors* → *rooms* (soft-delete), *customer_locations* (location_level ∈ BUILDING/FLOOR/ROOM + location_id).
- **Store:** *stores* (owner_user_id, vendor_id unique, sale_status, prep_minutes), *categories*, *menu_items* (price int64, status on/off), *option_groups/options/menu_item_options*, *combos/combo_items*, *ship_fee_rules* (scope building/floor/room + unit_fee), *operating_hours*, *operating_hours_change_requests*, *favorites* (user_id + target_type STORE/ITEM + target_id, unique — tham chiếu mềm, mục tiêu bị xoá tự rơi khỏi join).
- **Order:** *orders* (code unique, location_level, fulfillment, status [11 trạng thái], payment_method/status, grand_total, desired_time, pickup_pin), *order_items* (snapshot tên/giá/tuỳ chọn), *order_status_history*, *carts/cart_items* (1 cart/(khách,quán)), *outbox_events*, *processed_events*.
- **Payment:** *payments* (order_id XOR topup_id, idempotency_key unique), *wallets* (owner_type SYSTEM/CUSTOMER/STORE_PAYABLE), *ledger_entries* (sổ kép, amount có dấu, balance_after), *payout_batches*, *outbox/processed*.
- **Delivery:** *deliveries* (order_id unique, shipper_id nullable, status, batch_id), *delivery_batches* (≤5/shipper/quán), *delivery_incidents* (type/photo_url/status), *outbox/processed*.
- **Promotion:** *promotions* (type ORDER/SHIP_DISCOUNT, value_kind PERCENT/AMOUNT, usage_limit/used_count), *promotion_usages* (RESERVED→CONFIRMED→VOIDED).
- **Review:** *reviews* (target_type STORE/ITEM/SHIPPER, rating 1–5, *photo_urls jsonb*, status VISIBLE/HIDDEN), *review_replies*, *review_reports*, *outbox/processed*.
- **Reporting (read-model/CQRS):** *order_facts*, *store_performance* (avg_rating), *revenue_daily*, *user_growth_daily*, *processed*.
- **Notification:** *notifications* (channel push/email/in_app, read_at), *device_tokens* (FCM), *processed*.

Tham chiếu chéo service dùng UUID mềm (không FK liên-DB), ví dụ *stores.vendor_id* ↔ User, *ship_fee_rules.ref_id/orders.location_id* ↔ Location, *order_facts* dựng từ sự kiện.

Sổ cái kép (double-entry ledger) — Payment

Mỗi luồng tiền tạo hai bút toán cân bằng (tổng = 0) trong một transaction, lưu `balance_after` để truy vết; khoá FOR UPDATE trên cả hai ví (bên nợ và bên có) trong cùng transaction để chống chi vượt và chống race; chỉ tài khoản SYSTEM (UUID-0, vai trò clearing) được phép âm — số âm đó chính là nghĩa vụ phải trả lại cho khách/cửa hàng. (Mã: `services/payment/internal/usecase/ledger_helper.go`.)

Nghệp vụ	Bút toán
Thanh toán Ví	Nợ CUSTOMER → Có SYSTEM
Giao COD xong	Nợ SYSTEM → Có STORE_PAYABLE
Hoàn tiền (hủy)	Nợ SYSTEM → Có CUSTOMER
Chi trả cửa hàng (payout)	Nợ STORE_PAYABLE → Có SYSTEM

9. Nhất quán dữ liệu & giao dịch phân tán

- **Saga + sub-transaction barrier:** đặt hàng dùng chuỗi giao dịch cục bộ + bù trừ do DTM điều phối. Mỗi participant ghi một dòng chặn (`gid`, `branch_id`, `op`) vào bảng `dtm_barrier` trong cùng transaction với thao tác nghiệp vụ, nhờ đó chống được null-compensation (bù trừ cho một hành động chưa từng chạy, đồng thời chặn luôn hành động đến muộn sau khi đã bù trừ) và duplicate branch call (DTM gọi lại một nhánh đã thành công) — đảm bảo không trừ tiền/voucher mà đơn không tạo được và ngược lại.
- **Eventual consistency:** trạng thái giữa Order/Payment/Delivery/Reporting/Notification hội tụ qua sự kiện (thường < vài giây).
- **Idempotency:** consumer dùng `processed_events`; thao tác tiền idempotent theo `idempotency_key/order_id`.
- **Chống tranh chấp:** nhận đơn giao bằng `UPDATE ... WHERE shipper_id IS NULL` (CAS) → hai tài xế không nhận trùng; voucher dùng `FOR UPDATE` để siết `usage_limit`; payout loại đơn đã nằm trong batch PENDING/SETTLED để không trả gấp đôi.
- **Bù trừ bền vững (durable compensation):** nếu chính lời gọi bù trừ (Refund / ReleaseUsage) cũng thất bại, ý định rollback được ghi bên vào bảng `pending_compensations`; một worker nền quét mỗi 30 giây, tối đa 20 lần thử idempotent cho tới khi thành công. Đây là lời giải cho tình huống ‘bước rollback cũng có thể lỗi’ mà saga cơ bản bỏ ngỏ.
- **Dọn giữ chỗ mờ côi (orphaned reservation):** voucher được giữ ở trạng thái RESERVED khi áp; nếu saga chết giữa chừng (không Confirm mà cũng không Release), một janitor nền sẽ void các reservation quá hạn TTL 15 phút và trả lại `used_count` — chống rò rỉ hạn mức vĩnh viễn bằng bù trừ theo thời gian (timeout compensation).
- **Trả lại hạn mức cho voucher đã CONFIRMED:** khi huỷ hoặc từ chối một đơn đã đặt thành công, hệ void cả usage RESERVED lẫn CONFIRMED — tức bù trừ một bước đã commit, khác với việc chỉ huỷ một chỗ giữ tạm. Nếu không làm vậy, mỗi đơn bị huỷ sẽ rò rỉ vĩnh viễn một lượt dùng voucher.

10. Bảo mật & phân quyền (RBAC/JWT)

- **Xác thực:** JWT RS256, có chặn algorithm-confusion (bắt buộc SigningMethodRSA, từ chối token đổi alg sang HS256 để lợi dụng public key làm secret). Access token ngắn hạn + refresh token httpOnly cookie xoay vòng single-use; JTI được lưu whitelist trên Redis theo hướng fail-closed (Redis chết $\Rightarrow$ mọi token bị từ chối, không fail-open), cho phép thu hồi tức thì khi logout/suspend. Đăng ký xác thực bằng OTP 6 số, OTP được bcrypt-hash trước khi lưu (không lưu plaintext); khoá tài khoản 15 phút sau 5 lần đăng nhập sai. Hỗ trợ Google OAuth2.
- **RBAC:** mô hình role $\leftrightarrow$ permission có scope (global / vendor). 6 role (CUSTOMER, VENDOR_OWNER, VENDOR_STAFF, SHIPPER, ADMIN, SUPER_ADMIN) $\times$ 40 permission. Kiểm tra quyền tại tầng service qua gRPC CheckPermission / CheckVendorPermission về User service (nguồn sự thật RBAC duy nhất) — đảm bảo chủ quán chỉ thao tác đúng cửa hàng mình sở hữu. Kết quả kiểm quyền được cache-aside trên Redis (TTL 5 phút) và tự invalidate khi role/permission thay đổi.
- **Rate limiting tại gateway:** Kong giới hạn theo IP — 300 req/phút toàn cục, riêng /auth/* và đăng ký shipper chỉ 10 req/phút (chặn brute-force mật khẩu/OTP); client hiển thị thông báo thân thiện khi nhận 429.
- **Upload an toàn:** mọi điểm nhận ảnh (menu, avatar, KYC, sự cố, review) đều giới hạn 5MB và whitelist JPEG/PNG/WebP (*pkg/storage.AllowedImageType*) — chặn stored-XSS qua content-type tùy ý.
- **Ảnh KYC riêng tư:** giấy tờ shipper lưu ở bucket MinIO private (không đọc công khai); admin xem qua presigned URL hết hạn 15 phút. Ảnh menu/review dùng bucket public.
- **Audit log** cho thao tác đặc quyền; **không bao giờ lộ UUID thô** ra giao diện (luôn resolve sang tên/mã/đường dẫn).
- **Single-flight refresh phía client:** khi nhiều request cùng nhận 401, frontend gom về đúng một lời gọi /auth/refresh rồi mới retry các request đang chờ. Không có cơ chế này, refresh token single-use sẽ bị chính các request song song vô hiệu hoá lẫn nhau (request thứ hai dùng token vừa bị xoay vòng).

11. Quan sát, ghi log & chịu lỗi

- **Log tập trung (EFK):** log JSON slog + trace_id $\rightarrow$ Fluentd $\rightarrow$ Elasticsearch $\rightarrow$ Kibana. Có ILM policy tự xóa log sau 7 ngày và index template với mapping tường minh (trace_id, latency_ms, client_ip kiểu ip, caller_file/line/func), tự động provision Kibana data view lúc khởi động (docker/logging-init/init.sh).
- **Tracing phân tán (OpenTelemetry + Jaeger):** mọi service khởi tạo TracerProvider với exporter OTLP/gRPC (pkg/otel), instrument sẵn HTTP (otelgin) và gRPC cả phía server lẫn client (otelgrpc) ngay trong khung dùng chung pkg/app; bật/tắt qua config otel.endpoint mà không đổi code. Song song, Kong gán X-Trace-Id (correlation-id) lan truyền qua HTTP $\rightarrow$ gRPC $\rightarrow$ Kafka header, và trace_id này được gán làm attribute lên span OTel để đối chiếu chéo giữa log (Kibana) và trace (Jaeger).

- **Health/Readiness:** /health (liveness — process sống) tách khỏi /readyz (readiness — ping DB, Redis; trả 503 khi DB mất kết nối) để orchestrator không đẩy traffic vào service hỏng. Readiness cố ý KHÔNG probe Kafka: consumer đã tự retry, nên một nhịp lỗi broker không nên kéo cả service ra khỏi rotation.
- **Chịu lỗi:** circuit breaker (sony/gobreaker) per-target cho mọi gRPC nội bộ — mở mạch khi có ≥ 5 request trong cửa sổ 30s với $\geq 60\%$ lỗi hạ tầng, cho 3 probe ở trạng thái half-open rồi đóng lại sau 15s; chỉ lỗi hạ tầng (Unavailable, DeadlineExceeded) mới làm trip, lỗi nghiệp vụ (NotFound...) thì không — xếp lớp cùng timeout 5s và retry $\times 2$ trong cùng một chain (pkg/grpcx). Ngoài ra: outbox retry; consumer retry 3 lần + DLQ; idempotent consumer; degrade nhiều tầng (Redis/MinIO/gRPC downstream lỗi không làm sập service); panic recovery HTTP ghi log có cấu trúc kèm stack và trace_id; graceful shutdown (HTTP drain 15s $\rightarrow$ gRPC $\rightarrow$ consumer $\rightarrow$ DB/Redis); auto-migrate dùng advisory lock của Postgres nên nhiều replica khởi động song song không đưa migration.
- **Giám sát (Prometheus + Grafana):** mỗi service expose /metrics ngay từ khung dùng chung, với 4 metric: http_requests_total, http_request_duration_seconds (histogram), grpc_client_errors_total và circuit_breaker_state (0=closed / 1=half-open / 2=open). Prometheus scrape cả 10 service; Grafana đã cấu hình sẵn datasource.

12. Kiểm thử & đánh giá

- **Kiểm thử tự động:** go test ./... với Postgres + Redis đang chạy $\rightarrow$ 21 package ok, 0 FAIL, 264 test-case PASS, 0 SKIP. DB test được cô lập qua pkg/testutil.SetupTestDB (tạo <svc>_db_test riêng, auto-migrate, truncate giữa các test) nên không đụng dữ liệu dev. Lưu ý: integration test cần hạ tầng — chạy docker compose up -d postgres redis trước go test; nếu thiếu, chúng tự SKIP thay vì báo lỗi giả. go vet, tsc --noEmit và eslint đều sạch.
- **Test chứng minh cơ chế phân tán (không chỉ happy path):** place_order_compensation_test.go inject lỗi ở từng bước saga (Capture lỗi, ConfirmUsage lỗi, và cả trường hợp chính lời gọi bù trừ cũng lỗi) để kiểm chứng chuỗi bù trừ ngược và việc ghi pending_compensations; refresh_logout_test.go kiểm chứng replay refresh token đã xoay vòng bị từ chối 401; auth_login_test.go kiểm chứng khoá tài khoản sau 5 lần sai; outbox/envelope_version_test.go kiểm chứng envelope cũ (chưa có version) vẫn decode được; pkg/trace/*_test.go kiểm chứng lan truyền trace-id qua HTTP/gRPC/Kafka/AMQP.
- **Smoke scripts:** kịch bản REST end-to-end (register $\rightarrow$ verify $\rightarrow$ login $\rightarrow$ onboard).
- **E2E các tính năng mới (qua Kong + trình duyệt):** đã kiểm chứng — search unaccent + sort/lọc giá + OpenNow; yêu thích (PUT idempotent, đồng bộ tìm optimistic giữa danh sách/chi tiết/trang yêu thích); histogram sao [0,0,0,0,1] khớp dữ liệu + lọc theo sao; rate-limit trả header X-RateLimit-* và siết /auth/*; /readyz trả trạng thái DB/Redis; auto-migrate tự tạo bảng favorites khi service khởi động; giao diện mobile 390px và trạng thái chưa-đăng-nhập không lỗi console.
- **Dogfood E2E (qua trình duyệt, thực tế):** đã chạy trọn vòng đời cho cả 4 vai trò trên dữ liệu thật:

Admin tạo cây vị trí (Toà A/Tầng 1/P101) → khách đăng ký + OTP → chủ quán onboard + admin duyệt → admin tạo cửa hàng (tự cấp quyền quản lý) → chủ quán dựng menu + phí giao + giờ (admin duyệt) → khách đặt *VLX-260621-001* (COD, 60.000đ, giao P101) → chủ quán xác nhận → chuẩn bị → sẵn sàng → tài xế đăng ký (upload ảnh) + admin duyệt → nhận → lấy → đang giao → đã giao → đơn auto COMPLETED → khách đánh giá 5★ (cửa hàng + món + tài xế).

- **Kết quả:** xác nhận đúng cả luồng đồng bộ (saga + bù trừ) và bất đồng bộ (event-driven); idempotent consumer hoạt động; hoàn tiền khi huỷ; đối soát chi trả không trùng lặp. Kiểm chứng lại trên stack Docker đầy đủ (10 service + Kong + DTM): đặt đơn *VLX-260714-001* (ví điện tử + voucher) qua Kong — DTM ghi nhận giao dịch saga status=succeed với đúng ba nhánh ApplyPromotion → Capture → ConfirmUsage (nhánh bù trừ đã đăng ký nhưng không phải chạy); bảng dtm_barrier có 3 dòng chặn; số cái kếp cân bằng (tổng = 0), ví khách 500.000 → 423.000 và phản ánh đúng bên SYSTEM; voucher chuyển CONFIRMED, used_count = 1; sự kiện order.placed được publish và Notification lần Reporting đều đã tiêu thụ. Jaeger ghi một trace 9 span xuyên 5 service (order → store → location, order → promotion, order → payment).

13. Triển khai & hướng dẫn chạy

Cách 1 — toàn bộ bằng Docker (đơn giản nhất)

cd backend

make jwt-keys # sinh secrets/jwt_{private,public}.pem (RS256)

docker compose up -d # 10 service + Postgres/Redis/Kafka/DTM/MinIO/Kong

docker compose --profile monitoring up -d # Prometheus :17095, Grafana :17096, Jaeger :17093

Cách 2 — toàn bộ bằng go run (tiện debug)

docker compose up -d postgres redis kafka dtm minio kong

for s in user location store order promotion payment delivery review reporting notification; do

go run ./services/\$s/cmd & # cổng 17080..17089; Kong :17000

done

cd ../frontend && npm run dev # http://localhost:17070

LƯU Ý: không chạy TRỘN hai cách. config/.docker.yaml địa chỉ hoá DTM và*

các saga branch theo DNS của compose, còn config/.yaml dùng localhost — trộn*

hai chế độ sẽ khiến DTM không gọi được vào nhánh và saga gãy.

Mỗi service TỰ CHẠY MIGRATION khi start (auto-migrate, advisory lock nên nhiều

instance start song song vẫn an toàn). Migration thủ công: go run ./cmd/migrate <svc> up

Tài khoản seed: *admin@velox.test*, *super-admin@velox.test* (mật khẩu

Password123!); các vai trò khác tạo qua giao diện. Kiểm tra sẵn sàng: *curl*

localhost:1708x/readyz (trả trạng thái DB/Redis). Nếu DB được tạo lại giữa chừng, khởi động lại các service tiêu thụ sự kiện (order/payment/notification/reporting) để consumer gia nhập group đúng.

14. Đối chiếu yêu cầu đề bài

Yêu cầu bắt buộc	Tối thiểu	VeloxFood	Đạt
Số microservice	≥ 3	10	✓ vượt
RESTful API (CRUD, đúng tài nguyên/HTTP)	có	180 endpoint route qua Kong	✓
≥ 1 luồng nghiệp vụ dùng Message Queue	1	nhiều luồng qua Kafka + Outbox + DLQ	✓ vượt
Database riêng mỗi service	nên có	10 DB tách biệt, auto-migrate	✓
Xử lý lỗi (retry/log)	cơ bản	outbox retry, consumer retry + DLQ, gRPC timeout, idempotent, EFK, degrade	✓ vượt
Kiểm thử luồng liên service	có	21 package go test, 264 test-case PASS / 0 FAIL + E2E trình duyệt đủ 4 vai trò	✓
Service chạy độc lập + phối hợp	có	Docker + saga + sự kiện	✓

Yêu cầu nâng cao (cộng điểm)	Trạng thái
API Gateway	✓ Kong (khai báo declarative) là điểm vào duy nhất — 46 route, ẩn topology nội bộ. Mã: kong/kong.yml
Rate limiting tại gateway	✓ Hai tầng theo IP: 300 req/phút toàn cục; riêng /auth/* và đăng ký shipper chỉ 10 req/phút — chặn brute-force mật khẩu/OTP. Client hiển thị thông báo thân thiện khi nhận 429
Centralized logging	✓ EFK: log JSON (slog) kèm trace_id → Fluentd → Elasticsearch → Kibana. Có ILM tự xoá log sau 7 ngày và index template mapping tường minh (trace_id, latency_ms, client_ip kiểu ip)
Saga / event-driven	✓ Saga DTM cho hai luồng (đặt hàng — compensating; huỷ đơn — idempotent-retry) + Transactional Outbox → Kafka: 9 topic, 21 consumer group, mỗi topic kèm dead-letter
JWT & bảo mật xác thực	✓ RS256 có chặn algorithm-confusion; JTI whitelist trên Redis theo hướng fail-closed; refresh token xoay vòng single-use (có test chống replay); khoá tài khoản 15 phút sau 5 lần sai; OTP bcrypt-hash; Google OAuth2
Docker	✓ docker compose up -d dựng đủ 10 service + Postgres/Redis/Kafka/DTM/MinIO/Kong từ máy sạch (init script tự tạo 10 DB và bảng của DTM)
Service discovery	● Định tuyến tập trung qua Kong; mọi địa chỉ downstream (gRPC peer, DTM, saga branch, Kafka) đều config-driven — chuyển sang Kubernetes chỉ cần đổi DNS trong YAML, không sửa một dòng code. Chưa dùng registry động kiểu Consul/etcd
Retry policy	✓ Consumer Kafka retry 3 lần (backoff 0,5/1/2s) rồi đẩy sang <topic>.dlq kèm header truy vết; gRPC retry ×2 chỉ trên lỗi Unavailable, kèm deadline mặc định 5s; outbox dispatcher retry tới 5 lần
Health / readiness probe	✓ /health (liveness — process sống) tách khỏi /readyz (readiness — ping DB + Redis, trả 503 khi mất DB). Cố ý KHÔNG probe Kafka: consumer tự retry, một nhịp lỗi broker không nên kéo service khỏi rotation
Graceful shutdown	✓ Thứ tự có chủ đích: HTTP drain 15s → gRPC GracefulStop → consumer → flush span OTel → đóng Redis/DB

Yêu cầu nâng cao (cộng điểm)	Trạng thái
Tracing giữa service	✅ OpenTelemetry + Jaeger (exporter OTLP/gRPC), instrument sẵn HTTP (otelgin) và gRPC cả server lẫn client (otelgrpc). Đo được: một trace 9 span xuyên 5 service. Hạn chế tự nêu: ngữ cảnh trace chưa nối qua biên Kafka
Circuit breaker	✅ sony/gobreaker per-target: mở khi ≥ 5 request/30s và $\geq 60\%$ lỗi hạ tầng, half-open 3 probe, đóng lại sau 15s. Điểm tinh tế: chỉ lỗi hạ tầng (Unavailable, DeadlineExceeded) mới trip — lỗi nghiệp vụ thì không. Đo được: sau khi mạch mở, độ trễ giảm từ 5.055ms xuống 25ms
Dashboard giám sát	✅ Prometheus scrape 10/10 service; Grafana provisioned sẵn dashboard 4 panel: request rate, latency p95, lỗi gRPC client, circuit breaker state
Kubernetes	✗ Mới chỉ Docker Compose. Nhưng đã sẵn sàng: auto-migrate dùng advisory lock của Postgres nên nhiều replica khởi động song song không đưa migration

Vượt khung đề bài — các cơ chế phân tán đã hiện thực ngoài barem

Cơ chế	Vì sao đáng điểm	Mục
Sub-transaction barrier	Ghi một dòng chặn vào dtm_barrier trong CÙNG transaction với thao tác nghiệp vụ — chống null compensation (bù trừ cho hành động chưa từng chạy) và duplicate branch call (DTM gọi lại nhánh đã thành công)	§6
Bù trừ bền vững (durable compensation)	Trả lời câu hỏi mà saga cơ bản bỏ ngỏ: “nếu chính bước rollback cũng lỗi thì sao?”. Ý định rollback được ghi bền vào pending_compensations; worker nền quét mỗi 30 giây, tối đa 20 lần thử idempotent	§9
Bù trừ theo thời gian (timeout compensation)	Saga chết giữa chừng để lại voucher RESERVED mở còi; janitor nền void chúng sau TTL 15 phút và trả lại used_count — chống rò rỉ hạn mức vĩnh viễn	§9
Bù trừ một bước đã commit	Huỷ đơn đã đặt thành công vẫn void được usage CONFIRMED để trả quota — khác với việc chỉ huỷ một chỗ giữ tạm	§9
Idempotent receiver → hiệu ứng exactly-once	Kafka chỉ đảm bảo at-least-once. Ta đạt hiệu ứng đúng-một-lần bằng bảng processed_events (PK = event_id) ghi cùng transaction với business write. Kiểm chứng: tua offset consumer về 0, Kafka giao lại toàn bộ 8 sự kiện, số thông báo vẫn là 8 chứ không nhân đôi thành 16	§7.1
Bảo toàn thứ tự partition	Khi vừa không xử lý được vừa không đẩy được sang DLQ, consumer CỐ Ý không commit và dừng ở message đó — giữ thứ tự thay vì bỏ qua	§7.1
Versioning sự kiện	Envelope mang trường version; envelope cũ (chưa có version) vẫn decode được — cho phép tiến hoá schema sự kiện mà không phá vỡ consumer cũ. Có test riêng	§7.1
CQRS read model	Reporting là read model thuần, dựng 100% từ sự kiện, không có API ghi nghiệp vụ, DB riêng — chấp nhận nhất quán cuối để không làm chậm đường ghi	§4, §8
Sổ cái kép (double-entry)	Mỗi luồng tiền sinh hai bút toán cân bằng (tổng = 0) trong một transaction, khoá FOR UPDATE cả hai ví; chỉ tài khoản SYSTEM (clearing) được phép âm vì sổ âm chính là nghĩa vụ phải trả lại	§8
Fail-closed authorization	JTI dùng whitelist chứ không phải blacklist: Redis chết thì mọi token bị từ chối (fail-closed), không fail-open	§10

Cơ chế	Vì sao đáng điểm	Mục
Single-flight refresh (biên client)	Refresh token là single-use, nên N request cùng nhận 401 mà mỗi request tự gọi /auth/refresh sẽ tự vô hiệu hoá lẫn nhau. Frontend gom về đúng một lời gọi refresh rồi retry — bài toán điều phối đồng thời ở phía client	§10
Chống tranh chấp (concurrency control)	FOR UPDATE khi siết usage_limit của voucher; atomic claim (UPDATE ... WHERE shipper_id IS NULL) để hai tài xế không nhận trùng đơn; đối soát chỉ trả loại đơn đã nằm trong batch PENDING/SETTLED để không trả gấp đôi	§9

15. Hạn chế & hướng phát triển

- **Tracing qua biên Kafka:** span OpenTelemetry đã xuyên suốt HTTP và gRPC, nhưng ngữ cảnh trace (traceparent) chưa được inject vào header Kafka — span của consumer là span gốc mới, hiện chỉ liên kết với producer qua trace_id. Cần truyền traceparent để có waterfall liên mạch xuyên message queue.
- **Panic recovery cho gRPC:** phía HTTP đã có middleware recovery ghi log có cấu trúc (stack + method + path + trace_id); gRPC server chưa có interceptor recovery tương ứng.
- **DLQ tooling:** message vào <topic>.dlq hiện cần xử lý tay; nên có công cụ xem + replay.
- **Triển khai & giám sát:** đã có Prometheus (:17095) + Grafana (:17096, dashboard provisioned sẵn 4 panel) + Jaeger (:17093) trong Docker Compose qua profile monitoring; còn thiếu đóng gói Helm/Kubernetes.
- **Thanh toán MoMo:** sandbox mới tới payUrl (IPN cần địa chỉ công khai); realtime Firebase cần web key (hiện fallback polling).
- **RabbitMQ:** gói pkg/messaging/rabbitmq đã được hiện thực nhưng chưa wire vào service nào — toàn hệ hiện chỉ dùng Kafka; giữ lại cho hướng mở rộng (ví dụ hàng đợi tác vụ có ưu tiên).

16. Kết luận

VeloxFood là ứng dụng đặt đồ ăn hoàn chỉnh theo kiến trúc microservice, đáp ứng đầy đủ toàn bộ yêu cầu bắt buộc của Chủ đề 4 và phần lớn yêu cầu nâng cao. Hệ thống minh hoạ trọn vẹn các khái niệm cốt lõi của hệ phân tán đã học: phân rã theo miền + database-per-service, giao tiếp đồng bộ (REST/gRPC có deadline) song hành bất đồng bộ (Kafka), saga + bù trừ, transactional outbox + consumer at-least-once (retry/DLQ) + idempotent consumer, sổ cái kép, API Gateway có rate-limit, RBAC/JWT + lưu trữ riêng tư presigned, log tập trung + trace-id, liveness/readiness + graceful shutdown + auto-migrate — tất cả đã được kiểm chứng end-to-end qua cả 4 vai trò người dùng và bộ test tự động 18 package.

17. Phụ lục

17.1. Bảng trạng thái (enum) quan trọng

- **Order.status:** PENDING → CONFIRMED → PREPARING → READY/READY_PICKUP → SHIPPER_ASSIGNED → DELIVERING → DELIVERED → COMPLETED · (CANCELLED, REJECTED).
- **Order.payment_status:** UNPAID / PAID / REFUNDED.
- **Order.payment_method:** COD / MOMO / WALLET.
- **Delivery.status:** AVAILABLE → CLAIMED → PICKED_UP → DELIVERING → DELIVERED · (CANCELLED, STORE_DELIVERING).
- **Payment.status:** PENDING / CAPTURED / FAILED / REFUNDED.
Wallet.owner_type: SYSTEM / CUSTOMER / STORE_PAYABLE.
- **Promotion.usage:** RESERVED → CONFIRMED / VOIDED.
Review.target_type: STORE / ITEM / SHIPPER.

17.2. Catalog topic Kafka

- order.events
- payment.events
- wallet.events
- payout.events
- delivery.events
- store.events
- review.events
- vendor.events
- user.events

9 topic, mỗi topic kèm dead-letter <topic>.dlq (tự tạo khi có message hỏng đầu tiên); 21 consumer group.

17.3. Tham chiếu mã nguồn chính

- Saga đặt hàng: *services/order/internal/usecase/place_order_usecase.go*
- Sổ cái kép: *services/payment/internal/usecase/ledger_helper.go*
- Outbox: *pkg/outbox/outbox.go* · Consumer retry/DLQ: *pkg/messaging/kafka/consumer.go*
- gRPC client chuẩn (deadline): *pkg/grpcx/client.go* · Bootstrap (readyz/graceful/auto-migrate): *pkg/app/app.go*
- Event handlers: *services/*/internal/handler/event/*.go*
- Gateway routes + rate-limit: *backend/kong/kong.yml*
- **Saga DTM + sub-transaction barrier:**
pkg/saga/saga.go · *pkg/saga/barrier.go* · *services/order/internal/usecase/place_order_dtm.go* · *order_cancel_dtm.go*

- **Bù trừ bền vững & dọn giữ chỗ:**
services/order/cmd/wire_compensation_worker.go · services/promotion/cmd/wire_reservation_janitor.go
- **Chịu lỗi & quan sát:** Circuit breaker: pkg/grpcx/breaker.go · Metrics Prometheus: pkg/metrics/metrics.go · Tracing OTel: pkg/otel/otel.go · EFK bootstrap: docker/logging-init/init.sh